MODULE 26 · DOMAIN 4 - ASSEMBLING AND DEPLOYING APPLICATIONS

Memory and MCP

Giving an agent a memory that outlives each request, and a standard plug for its tools.

SECTION 4 of 6 EXAM WEIGHT 22% MODULE 26 of 40

admin@sthithapragna.com

A model's memory is a whiteboard wiped after every call. **Memory is the notebook you keep for it. MCP is the standard plug that lets it use any tool.**

The model itself remembers nothing

- Each model call is **stateless**. It sees only what is in that request.
- A conversation feels continuous only because **earlier turns are sent again**.
- Deployed agents run on several instances. The next turn may land on a different one.
- State kept in a Python variable is lost on restart, and invisible to other instances.

> ON THE EXAM
>
> Memory must live outside the agent process, in a datastore every instance can reach.

Short-term memory, or long-term memory?

SHORT-TERM

This conversation

- The turns so far in one thread
- Keyed by a thread or conversation ID
- Saved after each step by a checkpointer
- Ends when the conversation ends

LONG-TERM

Across conversations

- Facts and preferences about a user
- Keyed by user
- Extracted and stored deliberately
- Recalled weeks later, in a new thread

THE TIEBREAKER

Forgets within a conversation: short-term memory is missing. Forgets between conversations: long-term memory is missing.

Agent memory with Lakebase

ASPECT	DETAIL
The datastore	Lakebase: managed Postgres, built for fast reads and writes
Short-term	Conversation state checkpointed by thread ID
Long-term	A store of facts about each user, across sessions
Where	Agents on Databricks Apps, and on Model Serving in Public Preview
Access	Declared as a DatabricksLakebase resource, from module 25

ON THE EXAM

Lakebase is Databricks' answer to the guide's persistent datastore objective: an operational database, not an analytics table.

Which store holds what?

LAKEBASE

Operational state

- Small, frequent reads and writes
- One user or thread at a time
- Conversation state, preferences
- Low latency, row by row

AI SEARCH OR DELTA

Knowledge and analytics

- AI Search: documents found by meaning
- Delta: large tables for analysis
- Written in batches
- Not built for per-turn updates

THE TIEBREAKER

Memory that changes every turn belongs in Lakebase. Documents belong in AI Search. Analysis belongs in Delta.

Rules that keep memory useful and safe

- One **thread ID per conversation**. One **user ID** for long-term facts.
- Do not resend the whole history forever: **trim or summarise** older turns.
- Store only what the user would expect you to remember.
- Memory is data: **govern it**, protect it, and be able to delete it.
- Never let one user's memory load into another user's conversation.

> ON THE EXAM
>
> Long-term memory holds personal data. Treat it with the same care as any customer table.

How the Model Context Protocol works

01
Server
Exposes tools with names, descriptions, parameters.

02
Discover
The agent's client lists the available tools.

03
Choose
The model picks a tool, as in module 04.

04
Call
The client calls the server, which runs it.

05
Return
The result goes back into the conversation.

ON THE EXAM

Write a tool once as an MCP server, and any MCP-capable agent can use it.

Three kinds of MCP server

KIND	WHAT IT GIVES	HOW
Managed	AI Search, Genie, SQL, UC functions	Built in. Unity Catalog permissions enforced
External	Outside services such as GitHub or Slack	Through a Unity Catalog connection. Managed OAuth
Custom	Your own tools and logic	Your server, hosted as a Databricks App

ON THE EXAM

Managed for Databricks data, external for outside services, custom for your own logic. The guide names all three.

Managed, external, or custom?

MANAGED OR EXTERNAL

Use what exists

- Databricks data: managed servers
- Outside SaaS: external servers
- No server code to write
- Credentials handled by the platform

CUSTOM

Build your own

- Your own business logic
- Shared by many agents
- Hosted as a Databricks App
- You write and run the server

THE TIEBREAKER

Reach for managed or external first. Build a custom server only for logic that no existing server provides.

What else to know

- The **databricks-mcp** library provides clients for Databricks MCP servers.
- Managed server URLs follow a pattern, such as /api/2.0/mcp/ai-search/catalog/schema.
- Declare every resource behind a managed server when logging, from module 25.
- External credentials live in the **Unity Catalog connection**, never in code.
- Unity AI Gateway now also governs MCP traffic. External MCP Services are in Beta.

> ON THE EXAM
>
> Tools through MCP are still tools: module 04's rules on descriptions, order and safety all apply.

Harbourline's claims agent: memory and tools

01
Short-term
Lakebase checkpoints each claim chat.

02
Long-term
Preferred contact method, per customer.

03
Managed MCP
Policy index, claims Genie, UC functions.

04
External MCP
The CRM, through a UC connection.

05
Custom MCP
Fraud rules shared by three agents.

ON THE EXAM

Every piece reuses something governed: no memory in the process, and no keys in the code.

> A support agent deployed on Databricks Apps sometimes forgets what the user said two messages earlier in the same conversation.
>
> It stores the conversation history in a Python list inside the agent, and the app runs on several instances.

What should the engineer change?

A Checkpoint conversation state in Lakebase by thread ID

B Send only the latest user message to the model each turn

C Store each conversation as rows in an AI Search index

D Move to an LLM with a larger context window instead

◆ PAUSE AND DECIDE

What should the engineer change?

A Lakebase checkpoint by thread
Every instance reads the same saved state.

B Latest message only
Forgets everything, every turn.

C Conversations in AI Search
Built for finding documents, not per-turn state.

D Larger context window
The history never reaches that instance at all.

THE SIGNAL

Forgets within a conversation, a Python list, several instances. Short-term memory must move outside the process.

> A retail assistant should remember each customer's preferred store and language, and use them weeks later in a brand-new conversation.
>
> Short-term checkpointing is already working well.

What should the engineer add?

A Keep every past conversation in the prompt from now on

B Reuse the thread checkpoint from the last conversation

C A long-term memory store in Lakebase, keyed by user

D Fine-tune the model on each customer's past messages

◆ PAUSE AND DECIDE

What should the engineer add?

A Every conversation in the prompt

Grows without limit, and blows the context budget.

B Reuse the old thread

Mixes old conversations into new ones.

C Long-term store by user

Saves the facts themselves, recalled in any new thread.

D Fine-tune per customer

One model for millions of customers cannot hold each one's facts.

THE SIGNAL

Weeks later, in a brand-new conversation. Remembering across conversations is long-term memory, keyed by user.

> An agent needs a Genie Agent, an AI Search index and several Unity Catalog functions as tools.
>
> The team wants the quickest route that keeps Unity Catalog permissions enforced.

How should the agent reach these tools?

A Write a custom MCP server app that wraps each service

B Use the Databricks managed MCP servers for these resources

C Hard-code direct REST calls to each service in the agent

D Connect an external MCP server through a UC connection

◆ PAUSE AND DECIDE

How should the agent reach these tools?

A **Custom server app**
Builds what Databricks already provides.

B **Managed MCP servers**
Built in for exactly these resources, UC permissions enforced.

C **Hard-coded REST calls**
More code, and loses the standard tool interface.

D **External server via connection**
For outside services. These are Databricks resources.

THE SIGNAL

Genie, AI Search and UC functions are exactly what managed MCP servers expose. Use what exists.

> An engineering assistant must search the company's GitHub issues and its Atlassian wiki.
>
> Security wants governed access, OAuth handled by the platform, and no tokens in code.

Which approach fits?

A Store personal GitHub tokens inside the agent's code files

B Export the issues and wiki nightly into an AI Search index

C Build a custom MCP server that scrapes both of the sites

D External MCP servers reached through a UC connection

◆ PAUSE AND DECIDE

Which approach fits?

A Tokens in code

Breaks the stated security rule directly.

B Nightly export to AI Search

A day stale, and a copy outside the source's permissions.

C Custom scraping server

Builds, and maintains, what external servers provide.

D External MCP via UC connection

Governed, OAuth managed, no tokens in code.

THE SIGNAL

Outside services, governed, OAuth by the platform, no tokens. That is exactly what external MCP servers through a connection provide.

What to carry forward

- Models are stateless. **Memory lives outside the process**, in Lakebase.
- **Short-term**: this conversation, by thread ID. **Long-term**: facts across conversations, by user.
- Lakebase for per-turn state. AI Search for documents. Delta for analysis.
- MCP: **managed** for Databricks data, **external** for outside services, **custom** as an App.
- No memory in Python variables. No tokens in code.

> ON THE EXAM
>
> Next: module 27 covers CI/CD and the prompt lifecycle, including the Prompt Registry.